

Правительство Российской Федерации  
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ  
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ  
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ  
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»  
(НИУ ВШЭ)

Московский институт электроники и математики им. А.Н. Тихонова

ОТЧЕТ  
О ПРАКТИЧЕСКОЙ РАБОТЕ № 8\_  
по дисциплине «Криптографические методы защиты информации»  
ТЕМА РАБОТЫ

Студент гр. МКБ-251

\_\_\_\_\_ В.В. Созыкин

«\_\_» \_\_\_\_\_ 2026 г.

Руководитель

Заведующий кафедрой информационной  
безопасности киберфизических систем

канд. техн. наук, доцент

\_\_\_\_\_ О.О. Евсютин

«\_\_» \_\_\_\_\_ 2026 г.

Москва 2026

## СОДЕРЖАНИЕ

|                                                                   |    |
|-------------------------------------------------------------------|----|
| 1 Задание на практическую работу .....                            | 4  |
| 2 Краткие теоретические сведения .....                            | 5  |
| 2.1 Общие сведения об электронной подписи .....                   | 5  |
| 2.2 Стандарты электронной подписи и место ГОСТ Р 34.10–2012 ..... | 5  |
| 2.3 Алгоритмы ГОСТ Р 34.10–2012 .....                             | 6  |
| Формирование подписи .....                                        | 6  |
| 2.4 Операции на эллиптической кривой и модульная арифметика ..... | 6  |
| 3 Описание программной реализации .....                           | 7  |
| 3.1 Среда разработки и зависимости .....                          | 7  |
| 3.2 Параметры кривой и представление данных .....                 | 8  |
| 3.3 Теоретико-числовые функции (EGCD, обратный элемент) .....     | 8  |
| 3.3.1 Функция egcd(a, b) .....                                    | 8  |
| 3.3.2 Функция mod_inverse(a, m) .....                             | 8  |
| 3.4 Операции над точками эллиптической кривой .....               | 9  |
| 3.4.1 Функция es_add(p1, p2) .....                                | 9  |
| 3.4.2 Функция es_mult(k, point) .....                             | 9  |
| 3.5 Генерация ключевой пары .....                                 | 10 |
| 3.6 Формирование подписи .....                                    | 10 |
| 3.7 Проверка подписи .....                                        | 11 |
| 3.8 Инструкция по запуску программы .....                         | 11 |
| 3.9 Форматы файлов .....                                          | 12 |
| 3.10 Пользовательский интерфейс .....                             | 13 |
| 3.11 Ограничения и особенности реализации .....                   | 13 |
| 4 Результаты работы программы .....                               | 13 |
| Эксперимент 1 — Генерация ключей .....                            | 14 |
| Эксперимент 2 — Подпись файла .....                               | 16 |
| Эксперимент 3 — Проверка подписи (корректный случай) .....        | 18 |

|                                                                                 |    |
|---------------------------------------------------------------------------------|----|
| Эксперимент 4 — Проверка подписи после изменения файла (некорректный случай)... | 18 |
| 5 Выводы о проделанной работе.....                                              | 21 |
| 6 Список использованных источников.....                                         | 22 |
| Приложение А.....                                                               | 23 |
| Листинг программы.....                                                          | 23 |

## **1 Задание на практическую работу**

Целью практической работы является приобретение навыков программной реализации схем электронной подписи.

В рамках работы необходимо выполнить следующие задачи:

1. разработать программную реализацию схемы электронной подписи, соответствующей ГОСТ Р 34.10–2012, без использования готовых библиотечных средств реализации электронной подписи;
2. использовать готовую реализацию хэш-функции ГОСТ Р 34.11–2012;
3. обеспечить возможность генерации ключевой пары;
4. обеспечить возможность формирования электронной подписи для выбранного файла;
5. обеспечить возможность проверки ранее сформированной подписи;
6. подготовить отчёт по результатам выполнения практической работы.

Требования к разработанной программе:

1. программа должна принимать на вход файл, для которого выполняется формирование или проверка электронной подписи;
2. программа должна принимать на вход файл электронной подписи;
3. программа должна принимать на вход закрытый ключ подписи или открытый ключ проверки;
4. программа должна обеспечивать генерацию ключевой пары;
5. программа должна предоставлять пользователю выбор режима работы: формирование или проверка электронной подписи.

## **2 Краткие теоретические сведения**

### **2.1 Общие сведения об электронной подписи**

Электронная подпись представляет собой специальную битовую последовательность, формируемую на основе содержимого сообщения и секретного ключа подписанта. Основное назначение электронной подписи заключается в обеспечении:

1. контроля целостности передаваемых данных;
2. подтверждения подлинности отправителя;
3. подтверждения того, что сообщение было сформировано владельцем соответствующего закрытого ключа.

В асимметричных криптографических системах каждому пользователю соответствует пара ключей:

1. закрытый ключ, предназначенный для формирования подписи и хранимый в секрете;
2. открытый ключ, предназначенный для проверки подписи и доступный другим участникам обмена.

Перед формированием подписи, как правило, вычисляется хэш сообщения фиксированной длины. Подписывается не сам документ целиком, а его хэш-значение, что позволяет существенно сократить объём вычислений и обеспечить привязку подписи к содержимому документа.

### **2.2 Стандарты электронной подписи и место ГОСТ Р 34.10–2012**

В отечественной практике применялись различные стандарты электронной подписи. Основными из них являются:

1. ГОСТ Р 34.10–94, в котором электронная подпись строилась на основе арифметики в мультипликативной группе конечного поля;
2. ГОСТ Р 34.10–2001, в котором была реализована схема подписи на эллиптических кривых;
3. ГОСТ Р 34.10–2012, являющийся развитием предыдущего стандарта и также основанный на использовании группы точек эллиптической кривой.

Переход к эллиптическим кривым позволил обеспечить высокий уровень криптографической стойкости при меньших размерах ключей по сравнению с классическими схемами на конечных полях.

Стандарт ГОСТ Р 34.10–2012 предусматривает использование параметров размерности 256 или 512 бит. В настоящей работе рассматривается 256-битный вариант. В качестве хэш-функции применяется ГОСТ Р 34.11–2012 (Стрибог).

### 2.3 Алгоритмы ГОСТ Р 34.10–2012

Для работы схемы электронной подписи используются следующие параметры:

- 1)  $p$  — простое число, определяющее конечное поле;
- 2)  $a, b$  — коэффициенты уравнения эллиптической кривой;
- 3)  $q$  — порядок подгруппы точек;
- 4)  $P$  — базовая точка порядка  $q$ ;
- 5)  $d$  — закрытый ключ;
- 6)  $Q$  — открытый ключ;
- 7)  $H(M)$  — хэш-значение сообщения  $M$ .

Открытый ключ вычисляется по формуле:

$$Q = dP$$

#### Формирование подписи

Пусть  $M$  — подписываемое сообщение. Тогда процедура формирования подписи включает следующие шаги:

1. вычисляется хэш сообщения  $h = H(M)$ ;
2. определяется значение  $e = h \bmod q$ ; если  $e = 0$ , то принимается  $e = 1$ ;
3. выбирается случайное число  $k$ , удовлетворяющее условию  $1 \leq k \leq q-1$ ;
4. вычисляется точка  $C = kP$ ;
5. вычисляется значение  $r = x_C \bmod q$ ; если  $r = 0$ , выбирается новое  $k$ ;
6. вычисляется значение  $s = (rd + ke) \bmod q$ ; если  $s = 0$ , выбирается новое  $k$ ;
7. подписью является пара чисел  $(r, s)$ .

$$h = H(M)$$

$$e = h \bmod q$$

$$1 \leq k \leq q-1$$

$$C = kP$$

$$r = x_C \bmod q$$

$$s = (rd + ke) \bmod q$$

### 2.4 Операции на эллиптической кривой и модульная арифметика

Эллиптическая кривая над конечным полем задаётся уравнением:

$$y^2 \equiv x^3 + ax + b \bmod p$$

Основными операциями в группе точек эллиптической кривой являются:

1. сложение двух различных точек;

2. удвоение точки;
3. умножение точки на скаляр.

Для двух различных точек  $P1=(x_1,y_1)$  и  $P2=(x_2,y_2)$  коэффициент наклона определяется по формуле:

$$lambda = (y_2 - y_1) (x_2 - x_1)^{-1} mod p$$

Для случая удвоения точки используется формула:

$$lambda = (3x_1^2 + a) (2y_1)^{-1} mod p$$

Координаты результирующей точки вычисляются следующим образом:

$$x_3 = lambda^2 - x_1 - x_2 mod p$$

$$y_3 = lambda (x_1 - x_3) - y_1 mod p$$

Операция деления по модулю заменяется умножением на обратный элемент. Для нахождения обратного элемента используется расширенный алгоритм Евклида.

### 3 Описание программной реализации

#### 3.1 Среда разработки и зависимости

Программная реализация схемы электронной подписи выполнена на языке Python 3.

В работе использованы следующие программные средства:

1. стандартный модуль random, а именно SystemRandom, предназначенный для генерации криптографически стойких случайных чисел;
2. модуль gostcrypto.gosthash, применяемый для вычисления хэш-функции ГОСТ Р 34.11–2012 (Стрибог-256).

Следует подчеркнуть, что в рамках данной работы не использовались готовые библиотеки электронной подписи или готовые реализации операций на эллиптических кривых. Все основные этапы алгоритма, включая арифметику точек, генерацию ключей, формирование и проверку подписи, реализованы самостоятельно. Использование библиотеки ограничено только вычислением хэш-функции, что соответствует условиям задания.

#### 3.2 Параметры кривой и представление данных

В программе используются параметры эллиптической кривой для 256-битного варианта ГОСТ Р 34.10–2012. Заданы:

- 1) модуль конечного поля  $p$ ;
- 2) коэффициенты кривой  $a$  и  $b$ ;
- 3) порядок подгруппы  $q$ ;

4) базовая точка  $P = (P_x, P_y)$ .

Точки эллиптической кривой внутри программы представляются в виде кортежей  $(x, y)$ .

Нейтральный элемент группы, то есть точка на бесконечности, обозначается значением `None`.

Такой подход позволяет упростить реализацию операций сложения и умножения точек, а также сделать код более наглядным и удобным для анализа.

### 3.3 Теоретико-числовые функции (EGCD, обратный элемент)

Для выполнения арифметики на эллиптической кривой в программе реализованы вспомогательные функции теоретико-числового характера.

#### 3.3.1 Функция `egcd(a, b)`

Функция `egcd(a, b)` реализует расширенный алгоритм Евклида. В результате её выполнения определяются:

- 1) наибольший общий делитель чисел  $a$  и  $b$ ;
- 2) коэффициенты  $x$  и  $y$ , удовлетворяющие соотношению:

$$ax + by = \gcd(a, b)$$

Полученные значения используются для нахождения обратного элемента по модулю.

#### 3.3.2 Функция `mod_inverse(a, m)`

Функция `mod_inverse(a, m)` вычисляет число, обратное к  $a$  по модулю  $m$ , то есть значение  $a^{-1}$ , удовлетворяющее условию:

$$a \cdot a^{-1} \equiv 1 \pmod{m}$$

Алгоритм работы функции следующий:

1. число  $a$  приводится по модулю  $m$ ;
2. вызывается функция `egcd(a, m)`;
3. если  $\gcd(a, m) \neq 1$ , обратный элемент не существует;
4. в противном случае возвращается найденное значение.

Функция применяется при выполнении операций сложения и удвоения точек, а также при проверке электронной подписи.



### **3.4 Операции над точками эллиптической кривой**

#### **3.4.1 Функция `ec_add(p1, p2)`**

Функция `ec_add(p1, p2)` реализует операцию сложения двух точек эллиптической кривой.

В функции рассматриваются следующие случаи:

1. если первая точка является нейтральным элементом, возвращается вторая точка;
2. если вторая точка является нейтральным элементом, возвращается первая точка;
3. если точки являются взаимно противоположными, результатом является точка на бесконечности;
4. если точки совпадают, выполняется операция удвоения;
5. если точки различны, выполняется обычное сложение.

После вычисления коэффициента наклона определяются координаты новой точки  $(x_3, y_3)$ .

#### **3.4.2 Функция `ec_mult(k, point)`**

Функция `ec_mult(k, point)` реализует умножение точки на целое число методом Double-and-Add.

Алгоритм заключается в последовательном анализе двоичного представления числа  $k$ . На каждом шаге:

1. при необходимости текущая точка прибавляется к результату;
2. текущая точка удваивается;
3. число  $k$  сдвигается на один бит вправо.

Функция используется:

- 1) при генерации открытого ключа;
- 2) при вычислении точки в процессе формирования подписи;
- 3) при вычислении контрольной точки в процессе проверки подписи.

### **3.5 Генерация ключевой пары**

Генерация ключевой пары реализована в функции `generate_keys()`.

На первом этапе случайным образом выбирается закрытый ключ  $d$ , удовлетворяющий условию:

$$1 \leq d \leq q-1$$

После этого вычисляется открытый ключ по формуле:

$$Q = dP$$

Закрытый ключ сохраняется в файл `private.key` в шестнадцатеричном текстовом представлении.

Открытый ключ сохраняется в файл `public.key` в виде двух строк, содержащих координаты  $Q_x$  и  $Q_y$  в шестнадцатеричной форме.

### 3.6 Формирование подписи

Формирование подписи выполняется функцией `sign(msg_bytes, d)`.

На вход подаются:

- байтовое представление сообщения;
- закрытый ключ подписанта.

Процесс формирования подписи состоит из следующих этапов:

1. вычисляется хэш сообщения;

$$h = H(M)$$

2. полученное хэш-значение преобразуется в целое число и приводится по модулю  $q$ ;

$$e = h \bmod q$$

если  $e = 0$ , то  $e = 1$

3. при необходимости значение  $e$  заменяется на 1;
4. генерируется случайное значение  $k$ ;

$$1 \leq k \leq q-1$$

5. вычисляется точка  $C=kP$ ;

$$C = kP$$

6. определяется величина  $r=x_C \bmod q$ ;

$$r = x_C \bmod q$$

7. вычисляется величина  $s=(rd+ke) \bmod q$ ;

$$s = (r \cdot d + k \cdot e) \bmod q$$

8. подпись формируется как конкатенация 32-байтовых представлений значений  $r$  и  $s$ .

Итоговая длина подписи составляет 64 байта.

### 3.7 Проверка подписи

Проверка подписи выполняется функцией `verify(msg_bytes, signature, Q)`.

На вход функции подаются:

1. исходное сообщение;
2. подпись;
3. открытый ключ.

Процедура проверки включает следующие шаги:

- 1) проверка корректности длины подписи;
- 2) извлечение значений  $r$  и  $s$ ;
- 3) проверка диапазонов допустимых значений;

$$0 < r < q \text{ и } 0 < s < q$$

- 4) вычисление хэша сообщения;

$$h = H(M)$$

- 5) вычисление параметра  $e$ ;

$$e = h \bmod q, \text{ если } e = 0 \text{ то } e = 1$$

- 6) нахождение обратного элемента

$$v = e^{-1} \bmod q$$

- 7) вычисление коэффициентов  $z_1$  и  $z_2$ ;

$$z_1 = (s \cdot v) \bmod q$$

$$z_2 = (-r \cdot v) \bmod q$$

- 8) вычисление контрольной точки

$$C = z_1 P + z_2 Q$$

- 9) сравнение значения

$$R = x_C \bmod q \text{ с параметром } r.$$

Если выполняется равенство  $R = r$ , подпись считается корректной.

### 3.8 Инструкция по запуску программы

Для запуска разработанной программы необходимо установить интерпретатор Python 3 и библиотеку gostcrypto.

Проверка установленной версии Python выполняется командой:

**python --version**

или

**python3 --version**

Установка требуемой библиотеки выполняется командой:

**pip install gostcrypto**

или

**pip3 install gostcrypto**

После установки зависимостей файл программы `el_sign.py` должен находиться в рабочем каталоге.

Запуск программы выполняется командой:

**`python el_sign.py`**

или

**`python3 el_sign.py`**

После запуска пользователю предлагается консольное меню, в котором доступны следующие режимы работы:

1. генерация ключевой пары;
2. формирование электронной подписи;
3. проверка электронной подписи;
4. завершение работы.

При генерации ключей создаются файлы:

- 1) `private.key`;
- 2) `public.key`.

При подписании файла создаётся файл подписи с расширением `.sig`.

Возможные ошибки при запуске и использовании программы связаны с отсутствием зависимости `gostcrypto`, отсутствием файлов ключей, отсутствием подписываемого файла или повреждением входных данных.

### **3.9 Форматы файлов**

В программе используются следующие форматы файлов:

1. `private.key` — закрытый ключ в текстовом шестнадцатеричном представлении;
2. `public.key` — открытый ключ в виде двух строк с координатами точки;
3. `*.sig` — бинарный файл электронной подписи;
4. произвольный пользовательский файл, предназначенный для подписания или проверки.

Структура файла подписи имеет вид:

- 1) байты 0–31 — значение `r`;
- 2) байты 32–63 — значение `s`.

### **3.10 Пользовательский интерфейс**

Взаимодействие с пользователем реализовано в консольном режиме. Пользователю предлагается выбрать одно из следующих действий:

1. сгенерировать ключевую пару;
2. подписать файл;
3. проверить подпись;
4. завершить работу программы.

При выборе режима подписи программа считывает исходный файл и закрытый ключ, формирует подпись и сохраняет её в отдельный файл с расширением .sig.

При выборе режима проверки программа считывает исходный файл, подпись и открытый ключ, после чего выполняет проверку и выводит результат в консоль.

Для обработки ошибок чтения файлов и некорректных входных данных используются конструкции обработки исключений.

### **3.11 Ограничения и особенности реализации**

Разработанная программа носит учебный характер и предназначена для демонстрации принципов работы алгоритма электронной подписи по ГОСТ Р 34.10–2012.

Необходимо учитывать следующие ограничения:

1. программа не содержит специальных механизмов защиты от атак по побочным каналам;
2. закрытый ключ сохраняется в открытом текстовом виде, что недопустимо в промышленных системах;
3. корректность формирования подписи зависит от качества генерации случайного числа  $k$ ;
4. не реализованы все возможные дополнительные проверки параметров кри-вой и открытого ключа.

Несмотря на указанные ограничения, программа корректно демонстрирует основные этапы генерации ключей, формирования подписи и проверки её подлинности.

## **4 Результаты работы программы**

В ходе выполнения практической работы было проведено тестирование разработанной программы в основных режимах её функционирования: генерация ключей, формирование электронной подписи и проверка подписи. Дополнительно был рассмотрен случай изменения подписанного файла для демонстрации механизма контроля целостности.

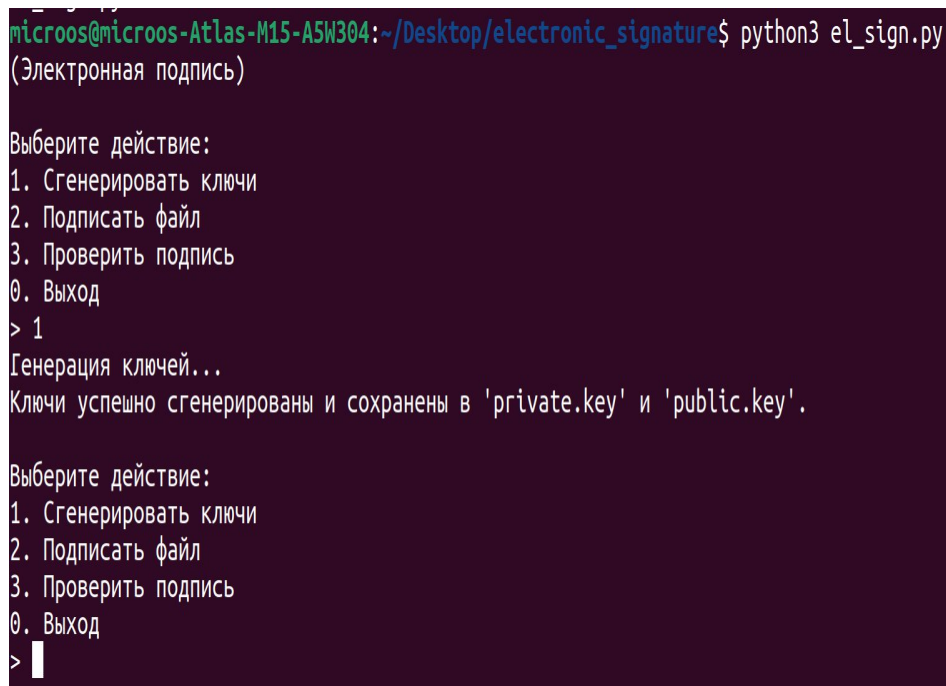
### **Эксперимент 1 — Генерация ключей**

На первом этапе был выбран режим генерации ключевой пары (рисунок 1).

В результате работы программы были сформированы и сохранены следующие файлы:

1. private.key, содержащий закрытый ключ;
2. public.key, содержащий координаты открытого ключа.

После завершения процедуры программа вывела в консоль сообщение об успешной генерации и сохранении ключей (рисунки 2, 3, 4).



```
microos@microos-Atlas-M15-A5W304:~/Desktop/electronic_signature$ python3 el_sign.py
(Электронная подпись)

Выберите действие:
1. Сгенерировать ключи
2. Подписать файл
3. Проверить подпись
0. Выход
> 1
Генерация ключей...
Ключи успешно сгенерированы и сохранены в 'private.key' и 'public.key'.

Выберите действие:
1. Сгенерировать ключи
2. Подписать файл
3. Проверить подпись
0. Выход
> █
```

Рисунок 1 - консольное меню

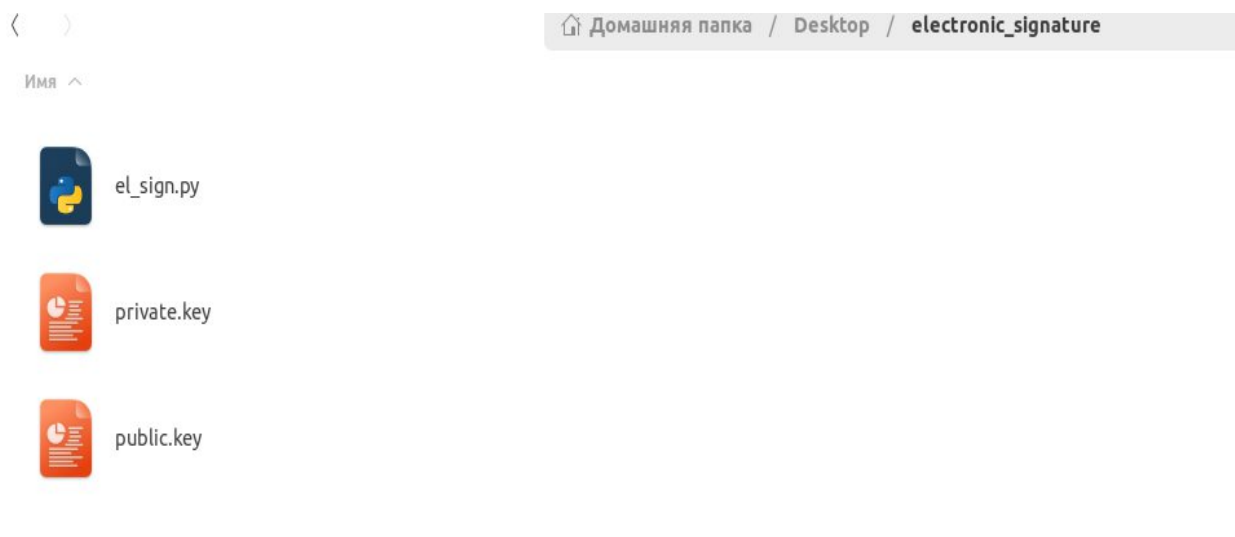


Рисунок 2 - сгенерированные файлы ключей в рабочем каталоге программы

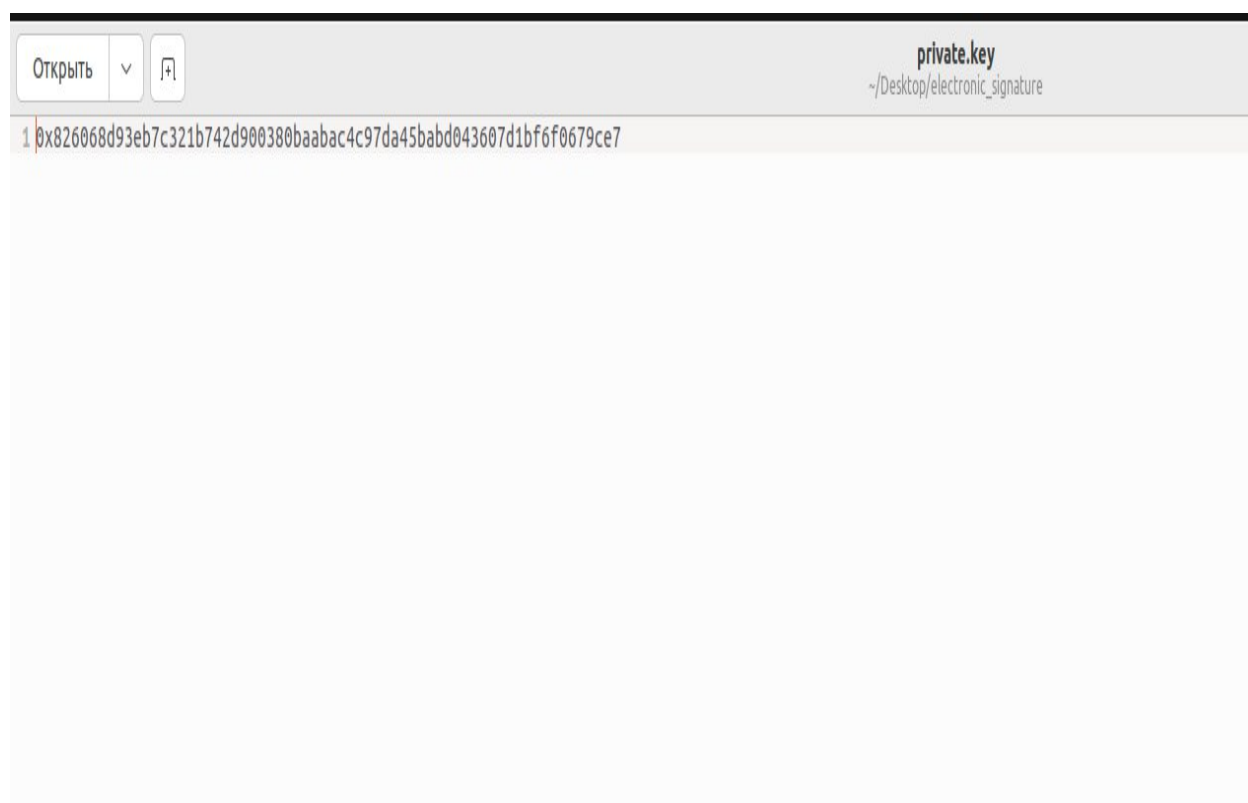


Рисунок 3 - содержимое файла private.key

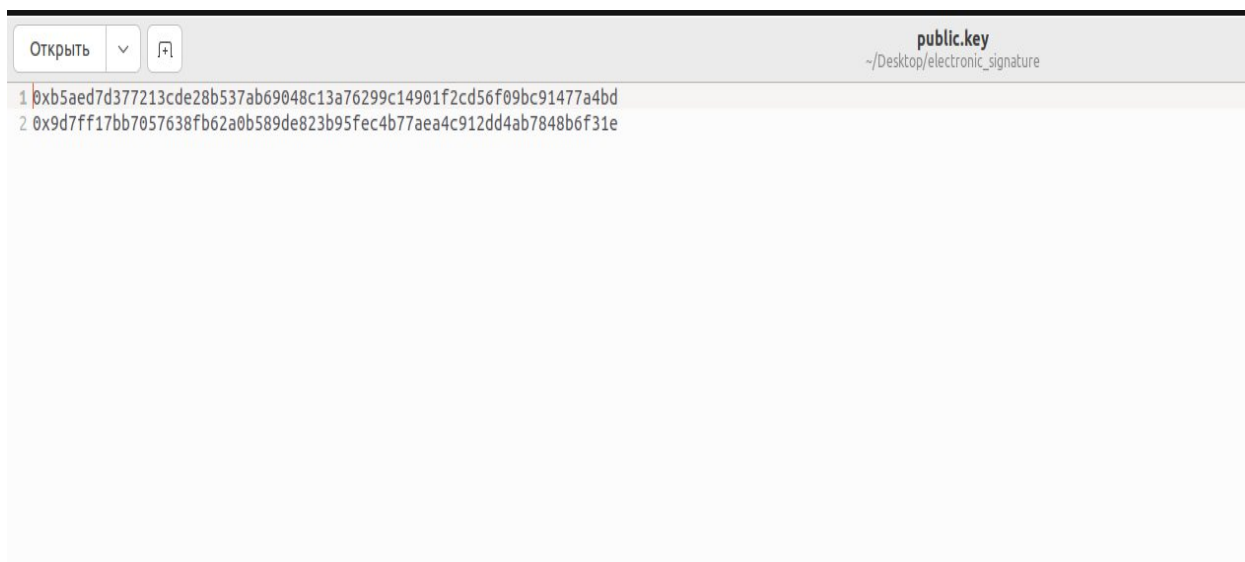


Рисунок 4 - содержимое файла public.key

## Эксперимент 2 — Подпись файла

На втором этапе был выбран режим формирования электронной подписи.

В качестве исходного файла использовался файл fables.txt (рисунок 7). После считывания содержимого файла и закрытого ключа программа вычислила электронную подпись и сохранила её в файл fables.txt.sig (рисунок 6), (рисунок 8).

По завершении операции в консоли было выведено сообщение об успешном подписании (рисунок 5).

В результате эксперимента подтверждено, что разработанная программа корректно выполняет процедуру вычисления подписи по заданному документу.

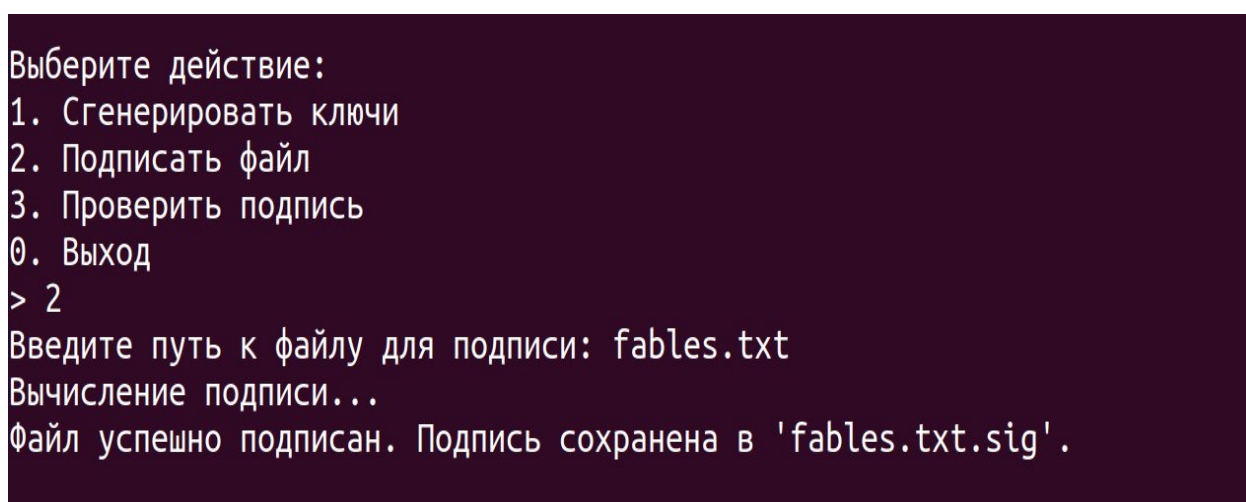


Рисунок 5 - процесс формирования электронной подписи файла





Рисунок 6 - файлы, полученные после подписания документа

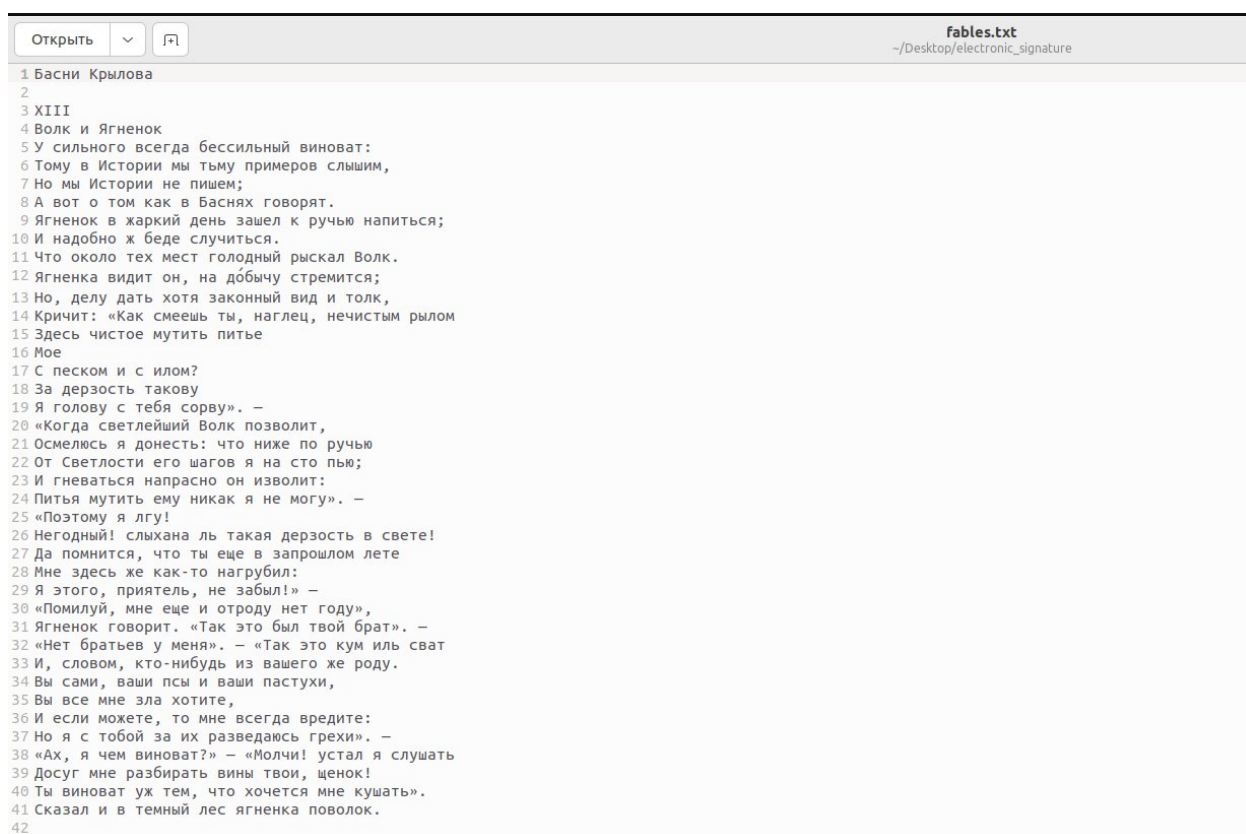


Рисунок 7 - фрагмент содержимого исходного файла fables.txt

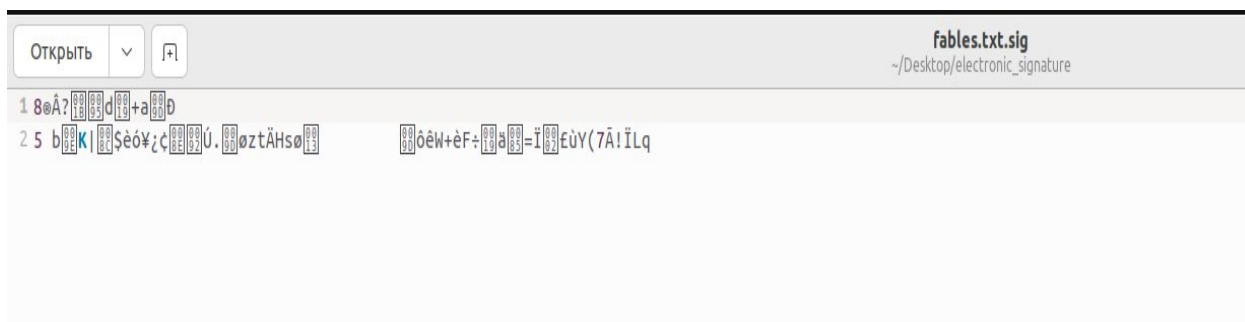


Рисунок 8 - содержимое файла электронной подписи fables.txt.sig

### Эксперимент 3 — Проверка подписи (корректный случай)

На третьем этапе был выбран режим проверки подписи.

Для проверки были указаны:

1. исходный файл fables.txt;
2. файл подписи fables.txt.sig;
3. открытый ключ public.key.

В результате выполнения проверки программа установила, что подпись соответствует содержимому файла и открытому ключу, после чего вывела сообщение о корректности подписи (рисунок 9).

Таким образом, подтверждено, что реализованный алгоритм правильно определяет подлинность подписанного документа в случае отсутствия изменений после подписания.

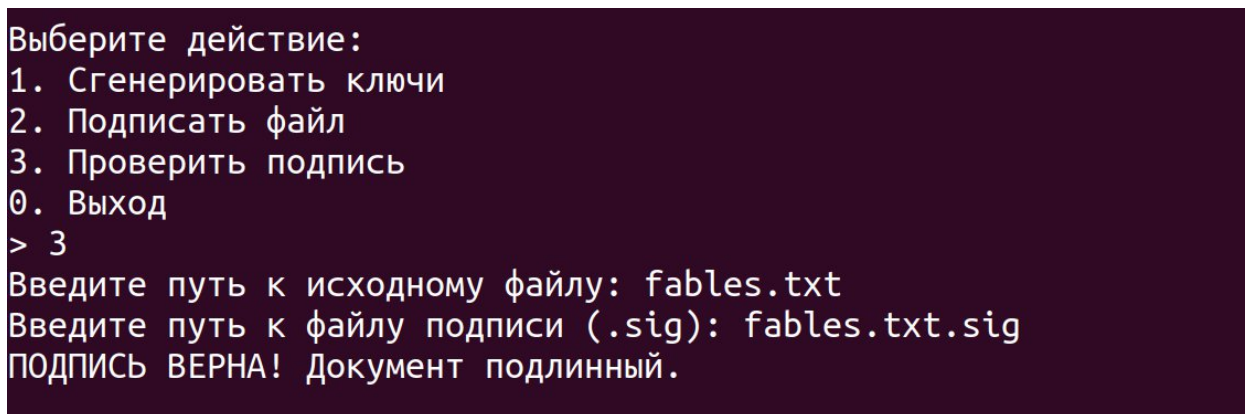


Рисунок 9 - результат проверки электронной подписи в корректном случае

### Эксперимент 4 — Проверка подписи после изменения файла (некорректный случай)

Целью данного эксперимента являлась демонстрация того, что изменение содержимого документа после подписания приводит к нарушению его целостности и делает исходную подпись недействительной.

Ход эксперимента был следующим:

1. после успешного подписания файла `fables.txt` содержимое файла было изменено (рисунки 10 и 11);
2. файл подписи `fables.txt.sig` оставался без изменений;
3. далее была выполнена проверка подписи для изменённого файла с использованием исходной подписи и открытого ключа.

В результате программа вывела сообщение о неверности подписи (рисунок 12).

Полученный результат объясняется тем, что даже незначительное изменение содержимого файла приводит к изменению хэш-значения сообщения. Следовательно, вычисляемые при проверке параметры уже не совпадают со значениями, использованными при формировании исходной подписи.

Таким образом, эксперимент подтверждает, что реализованная схема электронной подписи обеспечивает контроль целостности данных.

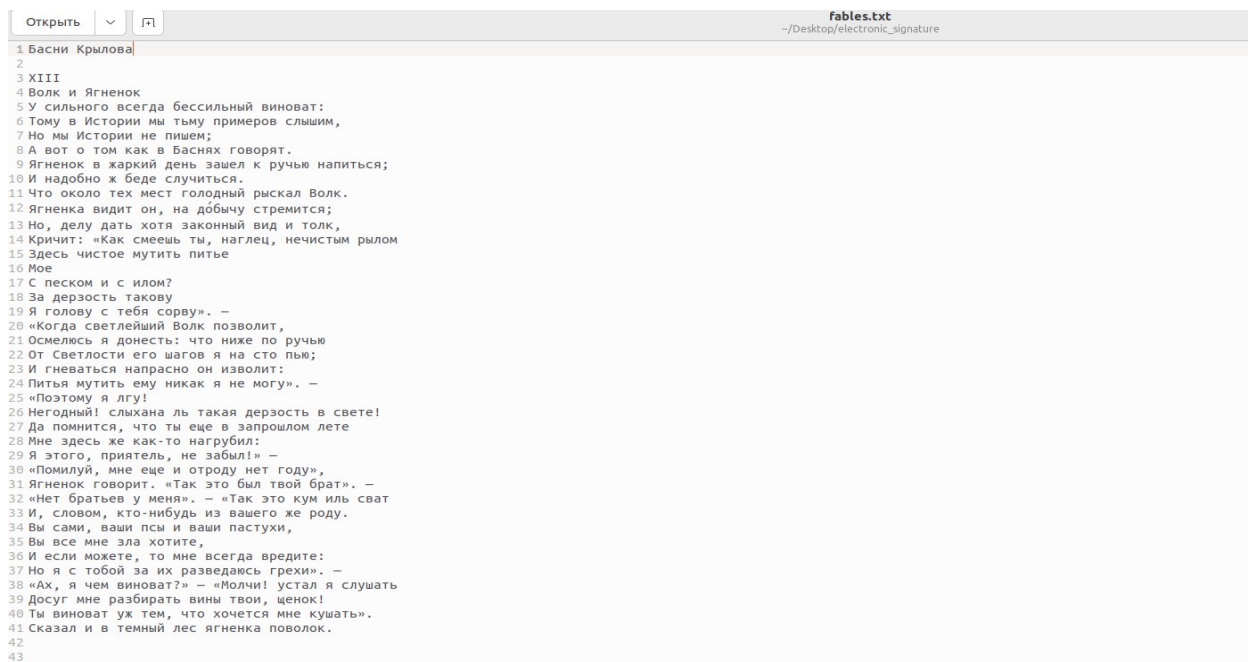
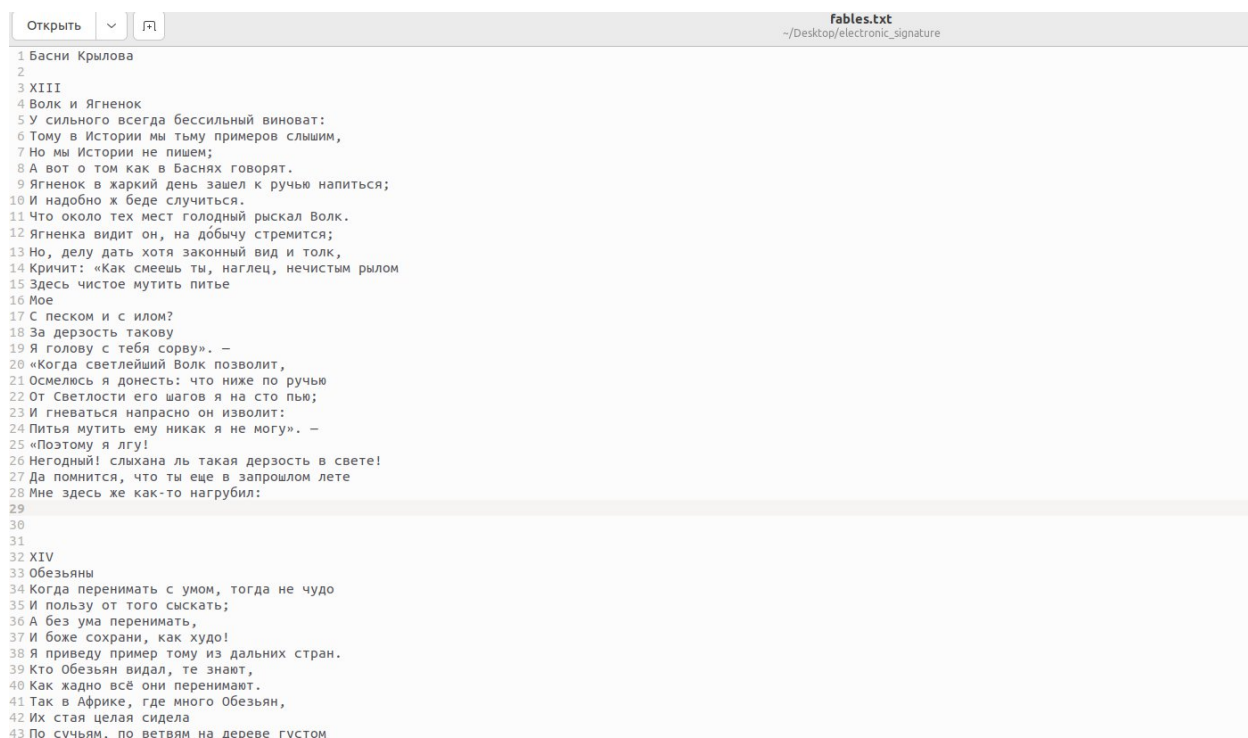


Рисунок 10 - исходное содержимое подписанного файла



```
1 Басни Крылова
2
3 XIII
4 Волк и Ягненок
5 У сильного всегда бессильный виноват:
6 Тому в Истории мы тому примеров слышим,
7 Но мы Истории не пишем;
8 А вот о том как в Баснях говорят.
9 Ягненок в жаркий день зашел к ручью напиться;
10 И надобно ж беде случиться.
11 Что около тех мест голодный рыскал Волк.
12 Ягненок видит он, на добычу стремится;
13 Но, делу дать хотя законный вид и толк,
14 Кричит: «Как смеешь ты, наглец, нечистым рылом
15 Здесь чистое мутить питье
16 Мое
17 С песком и с илом?
18 За дерзость такую
19 Я голову с тебя сорву». —
20 «Когда светлейший Волк позволит,
21 Осмелюсь я донести: что ниже по ручью
22 От Светлости его шагов я на сто пью;
23 И гневаться напрасно он изволил:
24 Питья мутить ему никак я не могу». —
25 «Поэтому я лгу!
26 Негодный! слыхана ль такая дерзость в свете!
27 Да помнится, что ты еще в прошлом лете
28 Мне здесь же как-то нагрубил:
29
30
31
32 XIV
33 Обезьяны
34 Когда перенимать с умом, тогда не чудо
35 И пользу от того сискать;
36 А без ума перенимать,
37 И боже сохрани, как худо!
38 Я приведу пример тому из дальних стран.
39 Кто Обезьян видал, те знают,
40 Как жадно всё они перенимают.
41 Так в Африке, где много Обезьян,
42 Их стая целая сидела
43 По сучьям, по ветвям на дереве густом
```

Рисунок 11 - изменённое содержимое файла после внесения правок



Рисунок 12 - результат проверки подписи после изменения файла

## **5 Выводы о проделанной работе**

В ходе выполнения практической работы была разработана программная реализация схемы электронной подписи по ГОСТ Р 34.10–2012 для 256-битного варианта параметров.

В процессе выполнения работы были реализованы:

1. операции сложения точек эллиптической кривой;
2. операция умножения точки на скаляр;
3. генерация закрытого и открытого ключей;
4. формирование электронной подписи;
5. проверка электронной подписи;
6. консольный пользовательский интерфейс для работы с файлами.
7. инструкция по запуску и использованию программы.

Проведённое тестирование показало, что программа корректно функционирует в основных режимах работы. В частности, подтверждена возможность успешной проверки подписи для неизменённого документа, а также выявления факта нарушения целостности после модификации файла.

Таким образом, поставленная цель работы достигнута: получены практические навыки реализации схемы электронной подписи на основе эллиптических кривых в соответствии с требованиями ГОСТ Р 34.10–2012.

## **6 Список использованных источников**

1. ГОСТ Р 34.10–2012. Информационная технология. Криптографическая защита информации. Процессы формирования и проверки электронной цифровой подписи.
2. ГОСТ Р 34.11–2012. Информационная технология. Криптографическая защита информации. Функция хэширования.
3. Методические указания к практической работе №8 по дисциплине «Криптографические методы защиты информации» .
4. Документация Python 3 по работе со случайными числами и файловым вводом-выводом – URL: <https://docs.python.org/3/library/random.html> (дата обращения 17.05.2026).
5. Документация используемого модуля gostcrypto – URL: <https://pypi.org/project/gostcrypto/> (дата обращения 10.05.2026).

## Приложение А

### Листинг программы

```
import random

from gostcrypto import gosthash

# 1. Параметры эллиптической кривой (ГОСТ Р 34.10-2012, 256 бит, ParamSetA)

p =
int('FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFD97',
16)

a =
int('FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFD94',
16)

b =
int('A7C5C5C9D749B45C0B7CE5F3A1CE2826AF96C61CC35AA15B9D19D0A805FF7DDE'
, 16)

q =
int('FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF6C611070995AD10045841B09B761B893', 16)

Px =
int('0000000000000000000000000000000000000000000000000000000000000001', 16)

Py =
int('8D91E471E0989CDA27DF505A453F2B7635294F2DDF23E3B122ACC99C9E9F1E14',
16)

P = (Px, Py) # Базовая точка

# 2. Математика эллиптических кривых и модульная арифметика

def egcd(a, b):
    """Расширенный алгоритм Евклида."""
    if a == 0:
        return (b, 0, 1)
    else:
        g, y, x = egcd(b % a, a)
        return (g, x - (b // a) * y, y)

def mod_inverse(a, m):
    """Нахождение обратного элемента по модулю."""
    a = a % m
    g, x, y = egcd(a, m)
```

```

    if g != 1:
        raise Exception(f'Обратного элемента не существует (НОД={g}). Модуль не является простым числом!')
    else:
        return x % m

def ec_add(p1, p2):
    """Сложение двух точек на эллиптической кривой."""
    if p1 is None: return p2
    if p2 is None: return p1

    x1, y1 = p1
    x2, y2 = p2

    if x1 == x2 and y1 != y2:
        return None # Точки симметричны, результат - бесконечно удаленная точка 0

    if x1 == x2:
        # Удвоение точки
        lam = (3 * x1 * x1 + a) * mod_inverse(2 * y1, p) % p
    else:
        # Сложение разных точек
        lam = (y2 - y1) * mod_inverse(x2 - x1, p) % p

    x3 = (lam * lam - x1 - x2) % p
    y3 = (lam * (x1 - x3) - y1) % p
    return (x3, y3)

def ec_mult(k, point):
    """Умножение точки на скаляр (Double-and-Add)."""
    result = None
    addend = point

    while k:
        if k & 1:

```



```

        result = ec_add(result, addend)
    addend = ec_add(addend, addend)
    k >>= 1
return result

```

### # 3. Алгоритмы ГОСТ

```
def generate_keys():
```

```
    """Генерация закрытого (d) и открытого (Q) ключей."""
```

```
    d = random.SystemRandom().randint(1, q - 1)
```

```
    Q = ec_mult(d, P)
```

```
    return d, Q
```

```
def sign(msg_bytes, d):
```

```
    """Формирование электронной подписи (ЭП)."""
```

```
    h_bytes = gosthash.new('streebog256', data=msg_bytes).digest()
```

```
    e = int.from_bytes(h_bytes, byteorder='big') % q
```

```
    if e == 0: e = 1
```

```
    while True:
```

```
        k = random.SystemRandom().randint(1, q - 1)
```

```
        C = ec_mult(k, P)
```

```
        r = C[0] % q
```

```
        if r == 0: continue
```

```
        s = (r * d + k * e) % q
```

```
        if s == 0: continue
```

```
    r_bytes = r.to_bytes(32, byteorder='big')
```

```
    s_bytes = s.to_bytes(32, byteorder='big')
```

```
    return r_bytes + s_bytes
```

```
def verify(msg_bytes, signature, Q):
```

```
    """Проверка электронной подписи."""
```

```

if len(signature) != 64:
    print("Ошибка: неверная длина подписи.")
    return False

r = int.from_bytes(signature[:32], byteorder='big')
s = int.from_bytes(signature[32:], byteorder='big')

if not (0 < r < q and 0 < s < q):
    return False

h_bytes = gosthash.new('streebog256', data=msg_bytes).digest()

e = int.from_bytes(h_bytes, byteorder='big') % q
if e == 0: e = 1

v = mod_inverse(e, q)
z1 = (s * v) % q
z2 = (-r * v) % q

C1 = ec_mult(z1, P)
C2 = ec_mult(z2, Q)
C = ec_add(C1, C2)

if C is None:
    return False

R = C[0] % q
return R == r

```

# 4. Интерфейс и работа с файлами

```

def main():
    print("Электронная подпись")
    while True:
        print("\nВыберите действие:")
        print("1. Сгенерировать ключи")
        print("2. Подписать файл")

```

```

print("3. Проверить подпись")
print("0. Выход")

choice = input("> ")

if choice == '1':
    print("Генерация ключей...")
    d, Q = generate_keys()
    with open('private.key', 'w') as f:
        f.write(hex(d))
    with open('public.key', 'w') as f:
        f.write(f'{hex(Q[0])}\n{hex(Q[1])}')
    print("Ключи успешно сгенерированы и сохранены в 'private.key' и
'public.key'.")

elif choice == '2':
    file_path = input("Введите путь к файлу для подписи: ")
    try:
        with open(file_path, 'rb') as f:
            msg = f.read()
        with open('private.key', 'r') as f:
            d = int(f.read().strip(), 16)

        print("Вычисление подписи...")
        sig = sign(msg, d)

        sig_path = file_path + ".sig"
        with open(sig_path, 'wb') as f:
            f.write(sig)
        print(f"Файл успешно подписан. Подпись сохранена в '{sig_path}'.")

    except Exception as e:
        print(f"[-] Ошибка при подписании: {e}")

elif choice == '3':

```

```

file_path = input("Введите путь к исходному файлу: ")
sig_path = input("Введите путь к файлу подписи (.sig): ")
try:
    with open(file_path, 'rb') as f:
        msg = f.read()
    with open(sig_path, 'rb') as f:
        sig = f.read()
    with open('public.key', 'r') as f:
        lines = f.readlines()
        Q = (int(lines[0].strip(), 16), int(lines[1].strip(), 16))

    is_valid = verify(msg, sig, Q)

    if is_valid:
        print("ПОДПИСЬ ВЕРНА! Документ подлинный.")
    else:
        print("ПОДПИСЬ НЕВЕРНА! Документ скомпрометирован.")

except Exception as e:
    print(f"[-] Ошибка при проверке: {e}")

elif choice == '0':
    break
else:
    print("Неверный выбор.")

if __name__ == '__main__':
    main()

```